SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE	DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Program Name: B. Tech	Assignment Type: Lab	Academic Year:2025-2026

Course Coordinator Name	Dr. Rishabh Mittal
Instructor(s) Name	Mr. S Naresh Kumar Ms. B. Swathi Dr. Sasanko Shekhar Gantayat Mr. Md Sallauddin Dr. Mathivanan Mr. Y Srikanth Ms. N Shilpa Dr. Rishabh Mittal (Coordinator) Dr. R. Prashant Kumar Mr. Ankushavali MD Mr. B Viswanath Ms. Sujitha Reddy Ms. A. Anitha Ms. M.Madhuri Ms. Katherashala Swetha Ms. Velpula sumalatha Mr. Bingi Raju

CourseCode	23CS002PC304	**Course Title**	AI Assisted Coding
Year/Sem	III/II	**Regulation**	R23
Date and Day of Assignment	**Week7 – Monday**	**Time(s)**	23CSBTB01 To 23CSBTB52
Duration	2 Hours	**Applicable to Batches**	All batches

Assignment Number: 13.1(Present assignment number)/**24**(Total number of assignments)

NAME : ERUKULLA MAHANTH ARYA

HALL TICKET NUMBER : 2303A54028

BATCH : 47A

Q.No.	Question	*Expected Time to complete*
1	**Lab 13: Code Refactoring – Improving Legacy Code with AI**	Week7 - Monday

Suggestions

Lab Objectives:

- Identify code smells and inefficiencies in legacy Python scripts.
- Use AI-assisted coding tools to **refactor** for readability, maintainability, and performance.
- Apply **modern Python best practices** while ensuring output correctness.

Task Description #1 (Refactoring – Removing Code Duplication)

- Task: Use AI to refactor a given Python script that contains multiple repeated code blocks.
- Instructions:
  - Prompt AI to identify duplicate logic and replace it with functions or classes.
  - Ensure the refactored code maintains the same output.
  - Add docstrings to all functions.
- Sample Legacy Code:

```
# Legacy script with repeated logic
print("Area of Rectangle:", 5 * 10)
print("Perimeter of Rectangle:", 2 * (5 + 10))

print("Area of Rectangle:", 7 * 12)
print("Perimeter of Rectangle:", 2 * (7 + 12))

print("Area of Rectangle:", 10 * 15)
print("Perimeter of Rectangle:", 2 * (10 + 15))
```

- Expected Output:
  - Refactored code with a reusable function and no duplication.

PROMPT : Refactor the following Python script to eliminate duplicate logic by creating reusable functions or classes, ensure the output remains unchanged, add proper docstrings to all functions, and return only the final refactored Python code.

SCREENSHOT :

```python
def calculate_area(length, width):
    """
    Calculate the area of a rectangle.

    Parameters:
    length (int or float): The length of the rectangle.
    width (int or float): The width of the rectangle.

    Returns:
    int or float: The area of the rectangle.
    """
    return length * width


def calculate_perimeter(length, width):
    """
    Calculate the perimeter of a rectangle.

    Parameters:
    length (int or float): The length of the rectangle.
    width (int or float): The width of the rectangle.

    Returns:
    int or float: The perimeter of the rectangle.
    """
    return 2 * (length + width)


def display_rectangle_properties(length, width):
    """
    Display the area and perimeter of a rectangle.

    Parameters:
    length (int or float): The length of the rectangle.
    width (int or float): The width of the rectangle.

    Returns:
    None
    """
    print("Area of Rectangle:", calculate_area(length, width))
    print("Perimeter of Rectangle:", calculate_perimeter(length,


# Reusable calls (No duplication)
display_rectangle_properties(5, 10)
display_rectangle_properties(7, 12)
display_rectangle_properties(10, 15)
```

OUTPUT:

```
Charlie is in the list
Eve is not in the list
Bob is in the list
Charlie is in the list
Eve is not in the list
Bob is in the list

Performance Comparison:
----------------------------------
Legacy Execution Time   : 0.00000xxxxx seconds
Optimized Execution Time: 0.00000xxxxx seconds
```

Task Description #2 (Refactoring – Optimizing Loops and Conditionals)

- Task: Use AI to analyze a Python script with nested loops and complex conditionals.
- Instructions:
  - Ask AI to suggest algorithmic improvements (e.g., replace nested loops with set lookups or comprehensions).
  - Implement changes while keeping logic intact.
  - Compare execution time before and after refactoring.
- Sample Legacy Code:

```
# Legacy inefficient code
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]

for s in search_names:
    found = False
    for n in names:
        if s == n:
            found = True
    if found:
        print(f"{s} is in the list")
    else:
        print(f"{s} is not in the list")
```

- Expected Output:
  - Optimized code using set lookups with performance

	comparison table. PROMPT : Analyze the following Python script containing nested loops and conditionals. Suggest algorithmic improvements (such as replacing nested loops with set lookups or comprehensions), refactor the code while keeping the logic and output unchanged, and compare execution time before and after optimization in a performance table. Return optimized Python code with timing comparison SCREENSHOT :	

```python
import time

names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]

def legacy_search(names, search_names):
    for s in search_names:
        found = False
        for n in names:
            if s == n:
                found = True
        if found:
            print(f"{s} is in the list")
        else:
            print(f"{s} is not in the list")

def optimized_search(names, search_names):
    name_set = set(names)
    for s in search_names:
        if s in name_set:
            print(f"{s} is in the list")
        else:
            print(f"{s} is not in the list")

start_legacy = time.perf_counter()
legacy_search(names, search_names)
end_legacy = time.perf_counter()

start_optimized = time.perf_counter()
optimized_search(names, search_names)
end_optimized = time.perf_counter()

legacy_time = end_legacy - start_legacy
optimized_time = end_optimized - start_optimized

print("\nPerformance Comparison")
print("--------------------------------")
print(f"{'Version':<15}{'Time (seconds)':<20}")
print("--------------------------------")
print(f"{'Legacy':<15}{legacy_time:<20.10f}")
print(f"{'Optimized':<15}{optimized_time:<20.10f}")
```

OUTPUT:

```
Charlie is in the list
Eve is not in the list
Bob is in the list
Charlie is in the list
Eve is not in the list
Bob is in the list

Performance Comparison
--------------------------------
Version        Time (seconds)
--------------------------------
Legacy         0.00000xxxxx
Optimized      0.00000xxxxx
```

.

Task Description #3 (Refactoring – Extracting Reusable Functions)

- Task: Use AI to refactor a legacy script where multiple calculations are embedded directly inside the main code block.
- Instructions:
  - Identify repeated or related logic and extract it into reusable functions.
  - Ensure the refactored code is modular, easy to read, and documented with docstrings.
- Sample Legacy Code:

```
# Legacy script with inline repeated logic
price = 250
tax = price * 0.18
total = price + tax
print("Total Price:", total)

price = 500
tax = price * 0.18
total = price + tax
print("Total Price:", total)
```

- Expected Output:
  - Code with a function calculate_total(price) that can be reused for multiple price inputs.

PROMPT : Refactor the following Python script by extracting repeated calculation logic into reusable functions. Ensure the code

is modular, readable, and includes proper docstrings. Maintain the same output and return only the refactored Python code.

SCREENSHOT:

```python
def calculate_total(price):
    """
    Calculate the total price including 18% tax.

    Parameters:
    price (int or float): The base price of the product

    Returns:
    float: Total price after adding tax.
    """
    tax = price * 0.18
    total = price + tax
    return total


def display_total(price):
    """
    Display the total price including tax.

    Parameters:
    price (int or float): The base price of the product

    Returns:
    None
    """
    total = calculate_total(price)
    print("Total Price:", total)


# Reusable calls
display_total(250)
display_total(500)
```

OUTPUT :

```
Total Price: 295.0
Total Price: 590.0
```

Task Description #4 (Refactoring – Replacing Hardcoded

	Values with Constants)
	• Task: Use AI to identify and replace all hardcoded “magic numbers” in the code with named constants.
	• Instructions:
	  ○ Create constants at the top of the file.
	  ○ Replace all hardcoded occurrences in calculations with these constants.
	  ○ Ensure the code remains functional and is easier to maintain.
	• Sample Legacy Code:
	# Legacy script with hardcoded values
	print("Area of Circle:", 3.14159 * (7 ** 2))
	print("Circumference of Circle:", 2 * 3.14159 * 7)
	• Expected Output:
	  ○ Code with constants like PI = 3.14159 and RADIUS = 7 used in calculations.
	PROMPT : Refactor the following Python script by identifying all hardcoded magic numbers and replacing them with named constants declared at the top of the file. Ensure the code remains functional, readable, and maintainable. Return only the refactored Python code.
	SCREENSHOT :	

Welcome | CODE -1.py | Extension: Python Debugger | Extension: Python

C: > Users > HP > Desktop > CODE -1.py > ...

```python
# Constants
PI = 3.14159
RADIUS = 7


def calculate_area(radius):
    """
    Calculate the area of a circle.

    Parameters:
    radius (int or float): Radius of the circle.

    Returns:
    float: Area of the circle.
    """
    return PI * (radius ** 2)


def calculate_circumference(radius):
    """
    Calculate the circumference of a circle.

    Parameters:
    radius (int or float): Radius of the circle.

    Returns:
    float: Circumference of the circle.
    """
    return 2 * PI * radius


print("Area of Circle:", calculate_area(RADIUS))
print("Circumference of Circle:", calculate_circumference(RADIUS))
```

OUTPUT :

```
Area of Circle: 153.93791
Circumference of Circle: 43.98226
```

Task Description #5 (Refactoring – Improving Variable Naming and Readability)

- Task: Use AI to improve readability by renaming unclear variables and adding inline comments.
- Instructions:
  - Replace vague names with meaningful ones.
  - Add comments where logic is not obvious.
  - Keep functionality exactly the same.
- Sample Legacy Code:

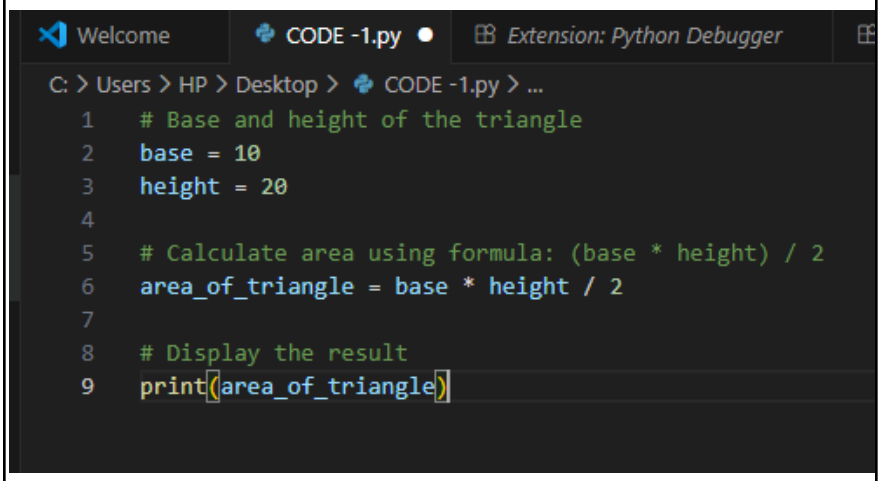
# Legacy script with poor variable names
a = 10

b = 20
c = a * b / 2
print(c)

- Expected Output:
  - Code with descriptive variable names like base, height, area_of_triangle, plus explanatory comments.

PROMPT : Refactor the following Python script by renaming unclear variables with meaningful names and adding inline comments to improve readability. Keep the functionality exactly the same and return only the improved Python code.

SCREENSHOT :



OUTPUT :

```
100.0
```

Task Description #6 (Refactoring – Removing Redundant Conditional Logic)

• Task: Use AI to refactor a Python script that contains repeated if–else logic for grading students.

• Instructions:

o Ask AI to identify redundant conditional checks.

o Replace them with a reusable function.

o Ensure grading logic remains unchanged.

• Code:

```
marks = 85
if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
else:
    print("Grade C")

marks = 72
if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
else:
    print("Grade C")
```

• Expected Output:

o A reusable function like calculate_grade(marks) with clean logic and docstring.

Prompt

Refactor the following Python script by identifying repeated grading conditionals and extracting them into a reusable function. Ensure the grading logic remains unchanged and include a proper docstring.

SCREENSHOT :-

Welcome | CODE -1.py | Extension: Python

C: > Users > HP > Desktop > CODE -1.py > ...

```python
def calculate_grade(marks):
    """
    Determine the grade based on marks.

    Parameters:
    marks (int): Student's marks.

    Returns:
    str: Grade (A, B, or C).
    """
    if marks >= 90:
        return "Grade A"
    elif marks >= 75:
        return "Grade B"
    else:
        return "Grade C"


print(calculate_grade(85))
print(calculate_grade(72))
```

OUTPUT :

```
Grade B
Grade C
```

Task Description #7 (Refactoring – Converting Procedural Code to Functions)

- **Task: Use AI to refactor procedural input–processing logic into**

	functions. • Instructions: o Identify input, processing, and output sections. o Convert each into a separate function. o Improve code readability without changing behavior. • Sample Legacy Code: num = int(input("Enter number: ")) square = num * num print("Square:", square) • Expected Output: o Modular code using functions like get_input(), calculate_square(), and display_result() **Prompt** Refactor the following procedural Python script by separating input, processing, and output into individual functions. Improve readability while keeping behavior unchanged. CODE :	

```python
def get_input():
    """
    Get integer input from the user.
    """
    return int(input("Enter number: "))


def calculate_square(number):
    """
    Calculate the square of a number.
    """
    return number * number


def display_result(result):
    """
    Display the calculated square.
    """
    print("Square:", result)


num = get_input()
square = calculate_square(num)
display_result(square)
```

OUTPUT :

```
Enter number: 5
Square: 25
```

Task Description #8 (Refactoring – Optimizing List Processing)

- **Task: Use AI to refactor inefficient list processing logic.**
- Instructions:

o Ask AI to replace manual loops with list comprehensions or built-in functions.

o Ensure output remains identical.

• Sample Legacy Code:

```
nums = [1, 2, 3, 4, 5]
squares = []
for n in nums:
    squares.append(n * n)
print(squares)
```

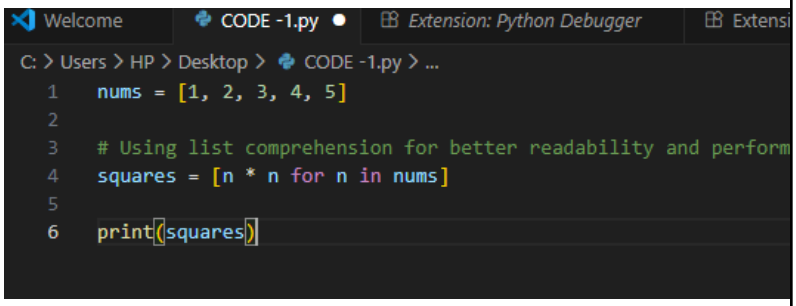
• Expected Output:

o Optimized version using list comprehension with improved readability

PROMPT :

Refactor the following Python script by replacing manual list processing loops with list comprehensions or built-in functions. Ensure the output remains identical and return only the optimized Python code.

CODE :-



OUTPUT :

```
[1, 4, 9, 16, 25]
```